Dodona 01 — cluster sanity

go / no-go, no load 2026-10-10 17:32 UTC

Objective

Does the cluster serve a real completion, and does it go THROUGH the control plane rather than past it?

Falsified by: orch_requests_total not moving, or a pool not answering

The promises everything later is scored against

SLO	target	direction
TTFT p95	1.5s	max
turn p95	30s	max
preemptions	0	max
shed rate at target load	0.05	max
concurrent sessions	derived in 02	min

From benchmarking/slos.py. The session target is derived in 02.

Concurrency target not measured yet — run 02-capacity.

1. Is anything listening?

PASS gateway /metrics

PASS gateway /v1/models

PASS prefill /metrics

PASS decode /metrics

>> All four endpoints answer, so the readings below are real.

2. Which pool does which job?

PASS gateway advertises: Qwen/Qwen2.5-7B-Instruct-AWQ

pool	running	waiting	prompt tok	gen tok
prefill	0	0	706,666	260
decode	0	0	706,567	516

prompt tok of 0 on a pool means nothing was ever routed to it.

PASS prefill emits 1.01 tokens per request (max_tokens=1 means ~1.0)

>> The split is real: prefill emitted 260 tokens over 258 requests — 1.01 each, which is the max_tokens=1 leg doing prompt work only, against decode's 516. But both pools show the same prompt-token count, so decode is still paying to process the prompt — section 5 measures why.

3. How much room does one pod have?

quantity	value	where it comes from
slice x util	10.80 GiB	HAMi gpumem x --gpu-memory-utilization
weights	5.20 GiB	Qwen2.5-7B AWQ int4, embeddings in fp16
activations + CUDA graphs	1.30 GiB	budget minus weights minus KV
KV cache	4.30 GiB	pod log: Available KV cache memory

PASS a pool of 4.9 is small enough for KV to bind

>> KV gets 4.30 of 10.80 usable GiB, which buys 4.9 full-length sequences per pod and a pool of 4.9. That is the scarcity the admission experiments need. The whole card would suggest 73.5 — do not quote that for a slice.

4. Does a request go through the gateway?

PASS orch_requests_total 258 -> 259

PASS completion in 0.324s

field	value
content	'ok'
via	local
latency	0.324s (tunnel included)

>> The counter moved and the model answered 'ok'. Guardrails, admission, placement and the hop all ran. via=local means our cluster served it rather than bursting to a provider.

5. Did any KV actually move?

PASS the router RECORDED a hop

field	value
src	prefill-0
dst	decode-0
req_id	http
tokens	16
backend	mooncake
prefix	802848469d1623df

a record, not evidence of bytes

where decode's prompt KV came from	tokens
computed from scratch	53,675
decode's own prefix cache	652,928
the connector (a real hop)	0

decode is SENT the whole prompt either way, so prompt_tokens_total never moves — only this split does.

connector counter	value
times asked	53,675
times it returned blocks	0

FAIL the connector has delivered at least one block

>> The connector was consulted 53,675 times and returned NOTHING. Decode recomputes every prompt. The 652,928 cache hits are decode's own prefix cache serving its own repeats, which is a different thing and is often mistaken for this one. The hop is bookkeeping.

6. Is anything scarce right now?

pool	kv_cache_usage_perc	vs 0.90
prefill	0.0000	below
decode	0.0000	below

>> Peak KV usage is 0.0% on an idle cluster, so nothing is scarce and the policy has nothing to decide. That is expected here. 02-capacity finds the load at which this crosses 90%, and until it does every admission arm would measure an inert component.

7. Latency on a small request

metric	seconds
n	10
min	0.321
p50	0.337
p95	0.346
max	0.346

>> p50 0.337s, p95 0.346s, spread 0.025s across ten requests, drifting -6% end to end. Stable. The tunnel is inside these numbers, so use make load to quote any.

8. Does decode pay for the prompt?

prompt tokens	p50 seconds
256	0.337
1,024	0.363
4,096	0.356
8,192	0.463
12,288	0.893

PASS all 5 prompt lengths completed

FAIL latency flat across prompt length (2.6x for 48x more prompt) — flat is what moved blocks would l

>> From 256 to 12,288 prompt tokens (48x), p50 went 0.337s to 0.893s (2.6x). Decode is paying for the prompt, which agrees with section 5: no blocks moved.

9. Grafana

vLLM — KV, running/waiting, TTFT, prefix cache

http://127.0.0.1:31495/d/dodona-vllm?from=now-30m&to=now

Gateway — admit, shed, place, latency by stage

```
http://127.0.0.1:31495/d/dodona-gateway?from=now-30m&to=now
```

Router — pick, sticky, KV hop

```
http://127.0.0.1:31495/d/dodona-router?from=now-30m&to=now
```

KEDA — scaler value against replicas

```
http://127.0.0.1:31495/d/dodona-keda?from=now-30m&to=now
```

HAMi — device memory against the slice cap

```
http://127.0.0.1:31495/d/dodona-hami?from=now-30m&to=now
```

explore: KV usage per pod

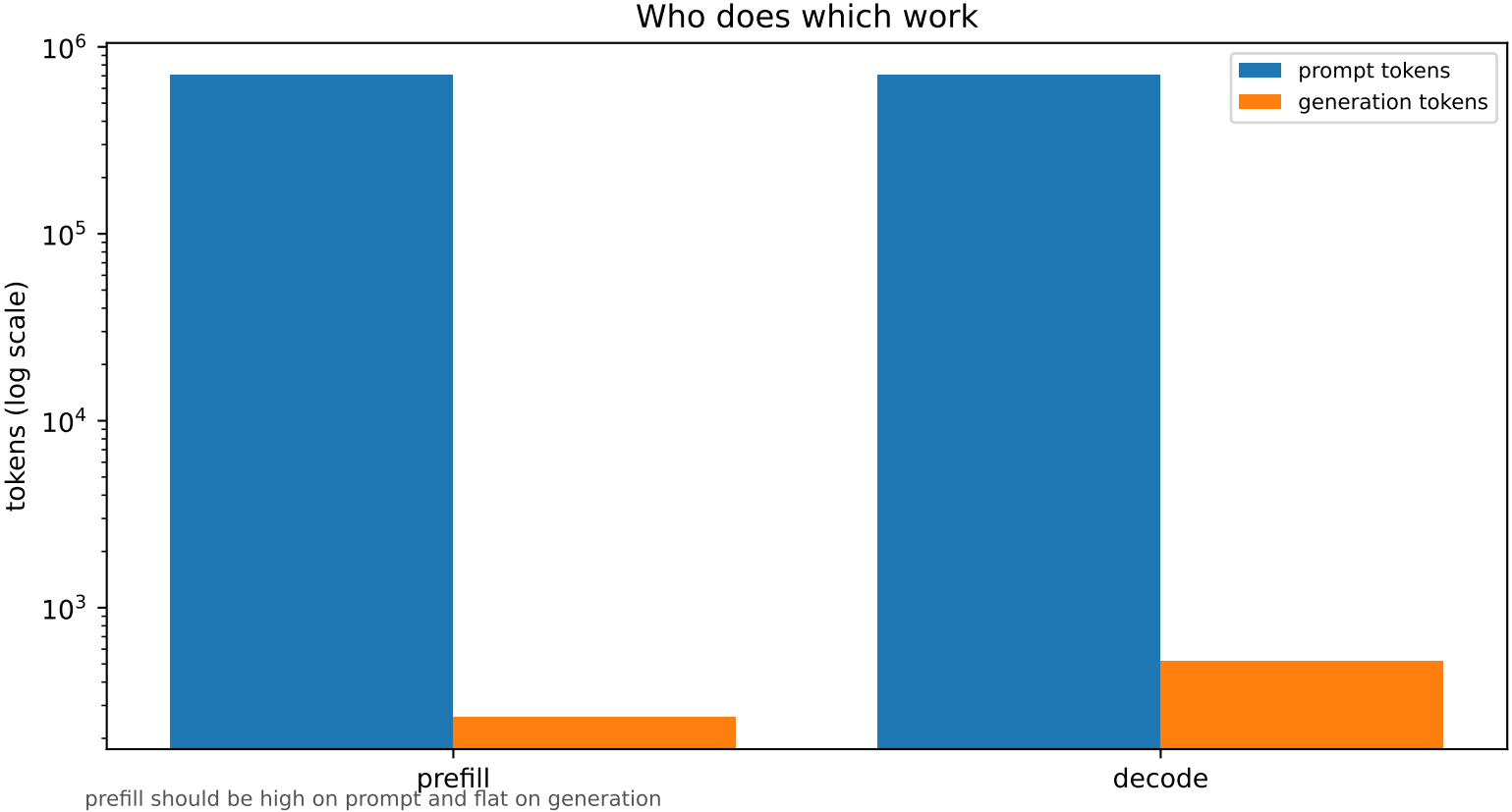
```
http://127.0.0.1:31495/explore?left=%7B%22datasource%22%3A%20%22Prometheus%22%2C%20%22queries%22%3A%20%5B%7B%22
```

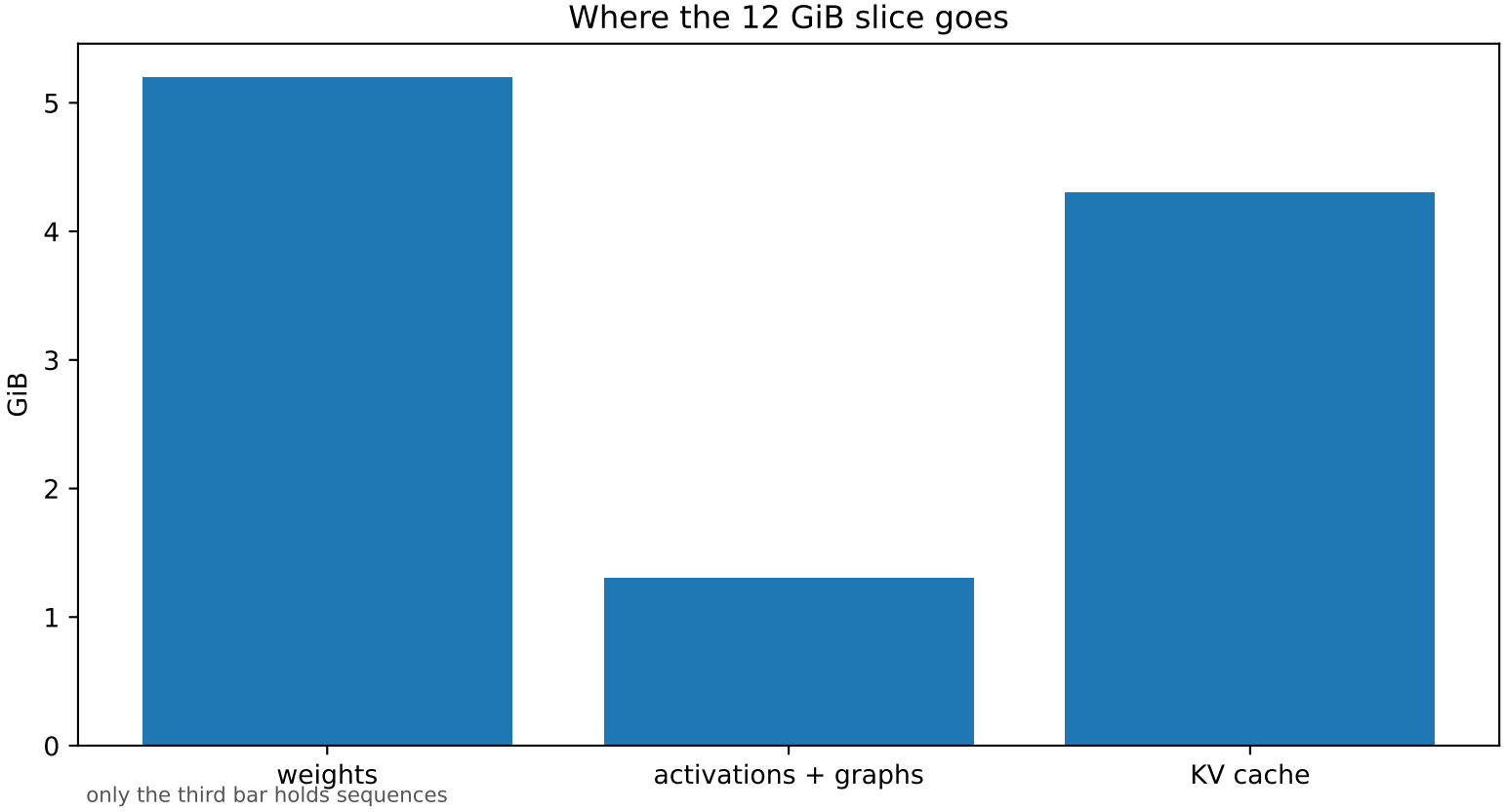
explore: queue depth per pod

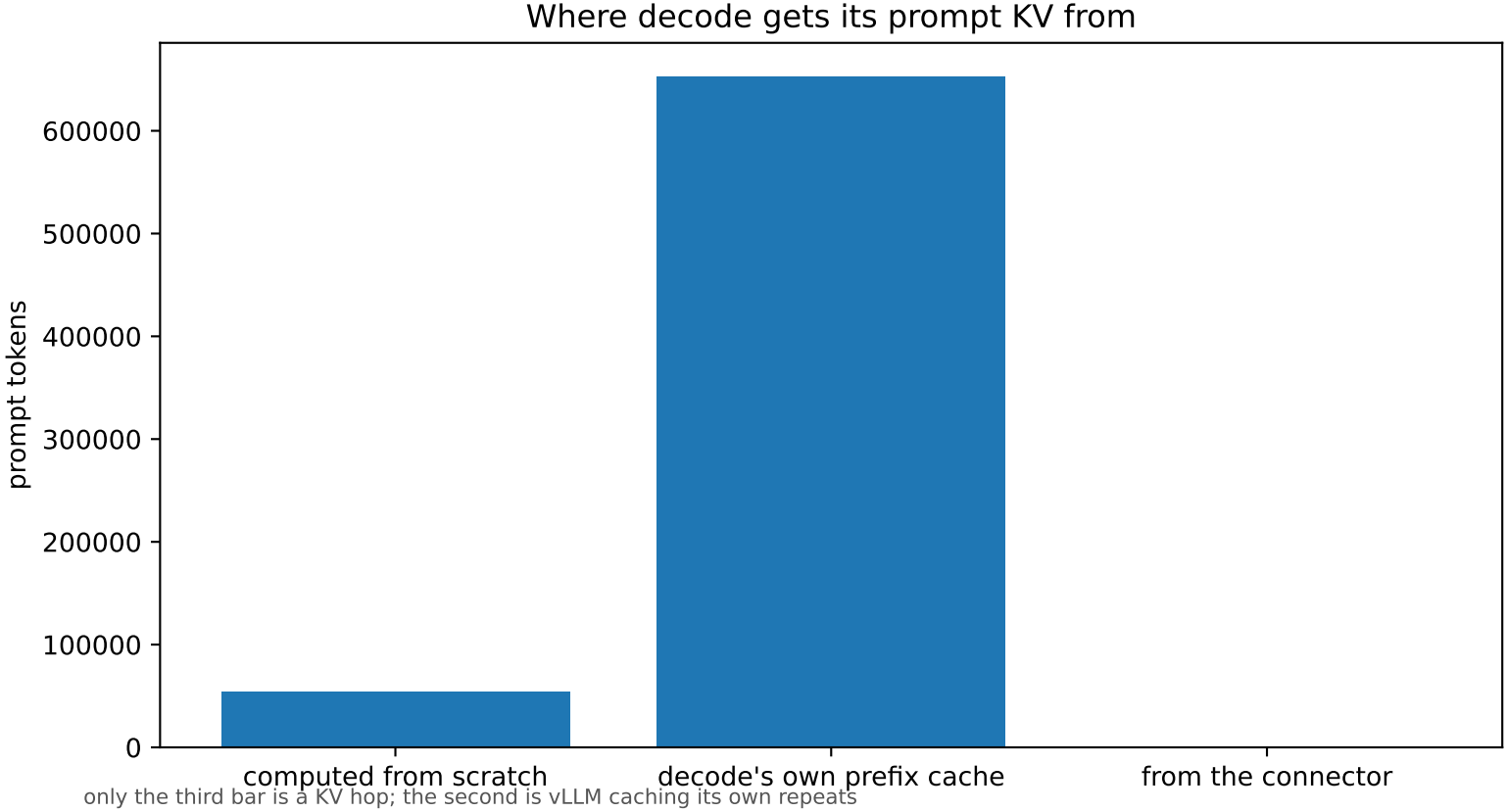
```
http://127.0.0.1:31495/explore?left=%7B%22datasource%22%3A%20%22Prometheus%22%2C%20%22queries%22%3A%20%5B%7B%22
```

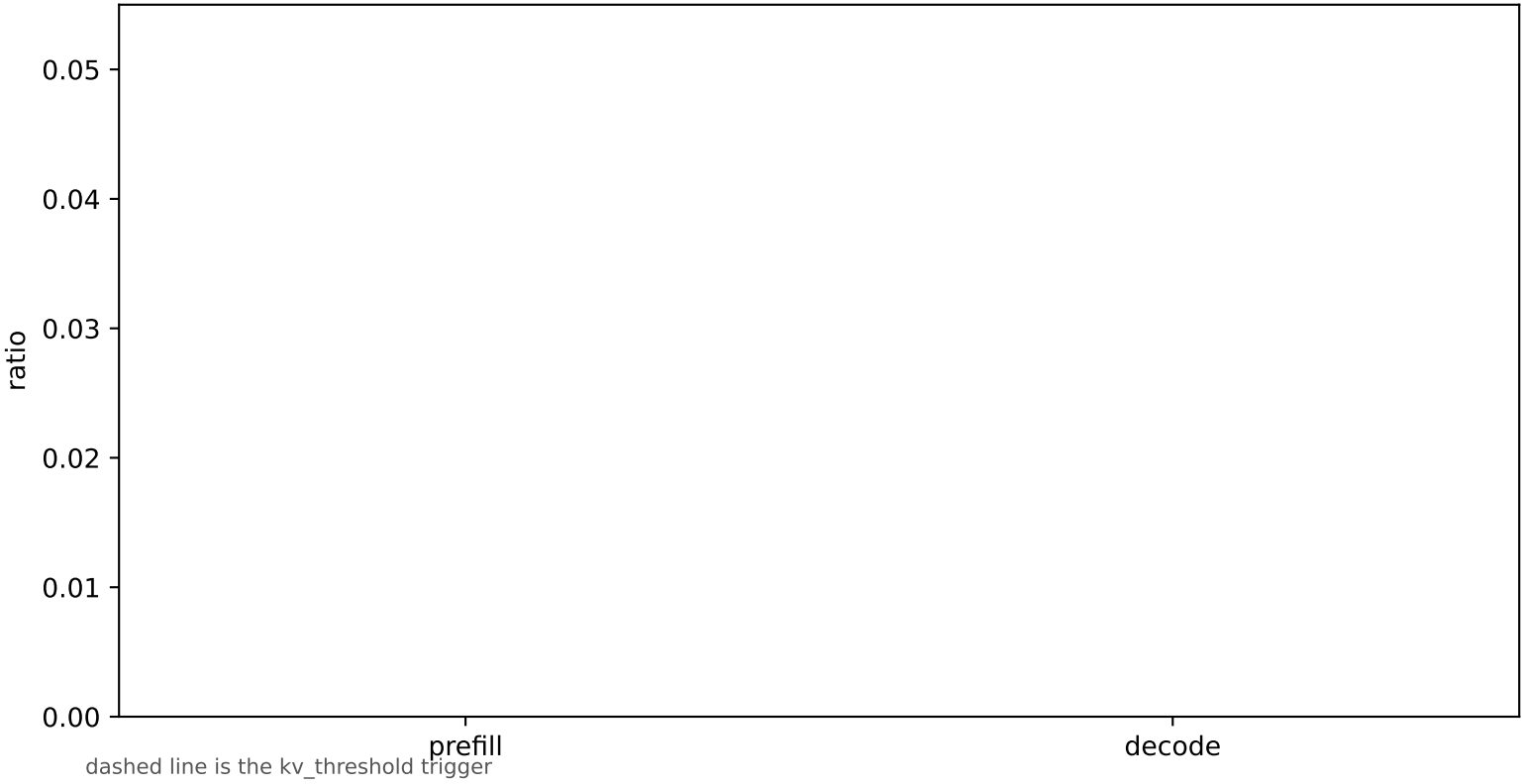
explore: KV actually transferred

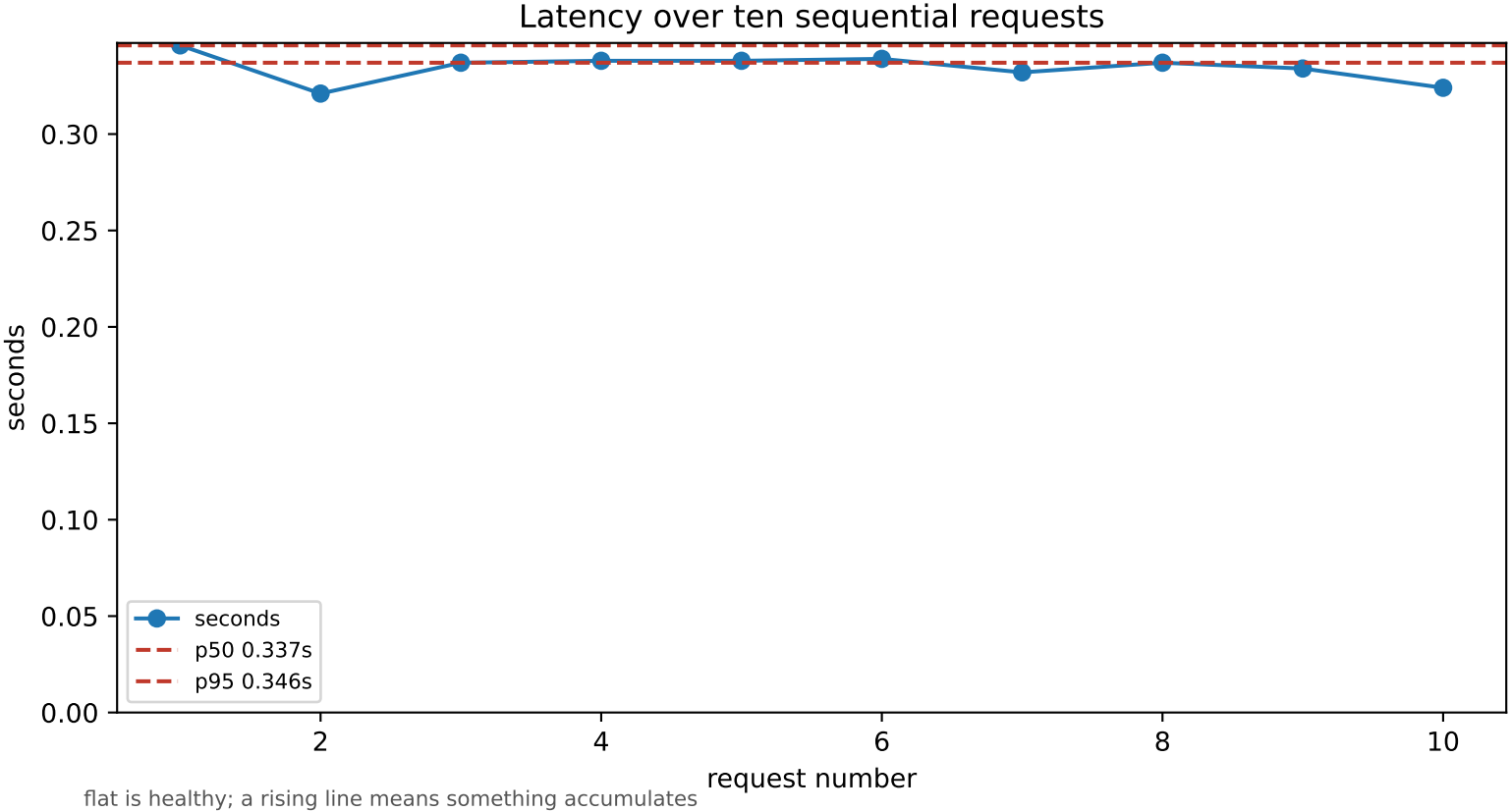
```
http://127.0.0.1:31495/explore?left=%7B%22datasource%22%3A%20%22Prometheus%22%2C%20%22queries%22%3A%20%5B%7B%22
```


Who does which work
tokens (log scale)
10^6
10^5
10^4
10^3
prompt tokens
generation tokens
prefill
decode
prefill should be high on prompt and flat on generation

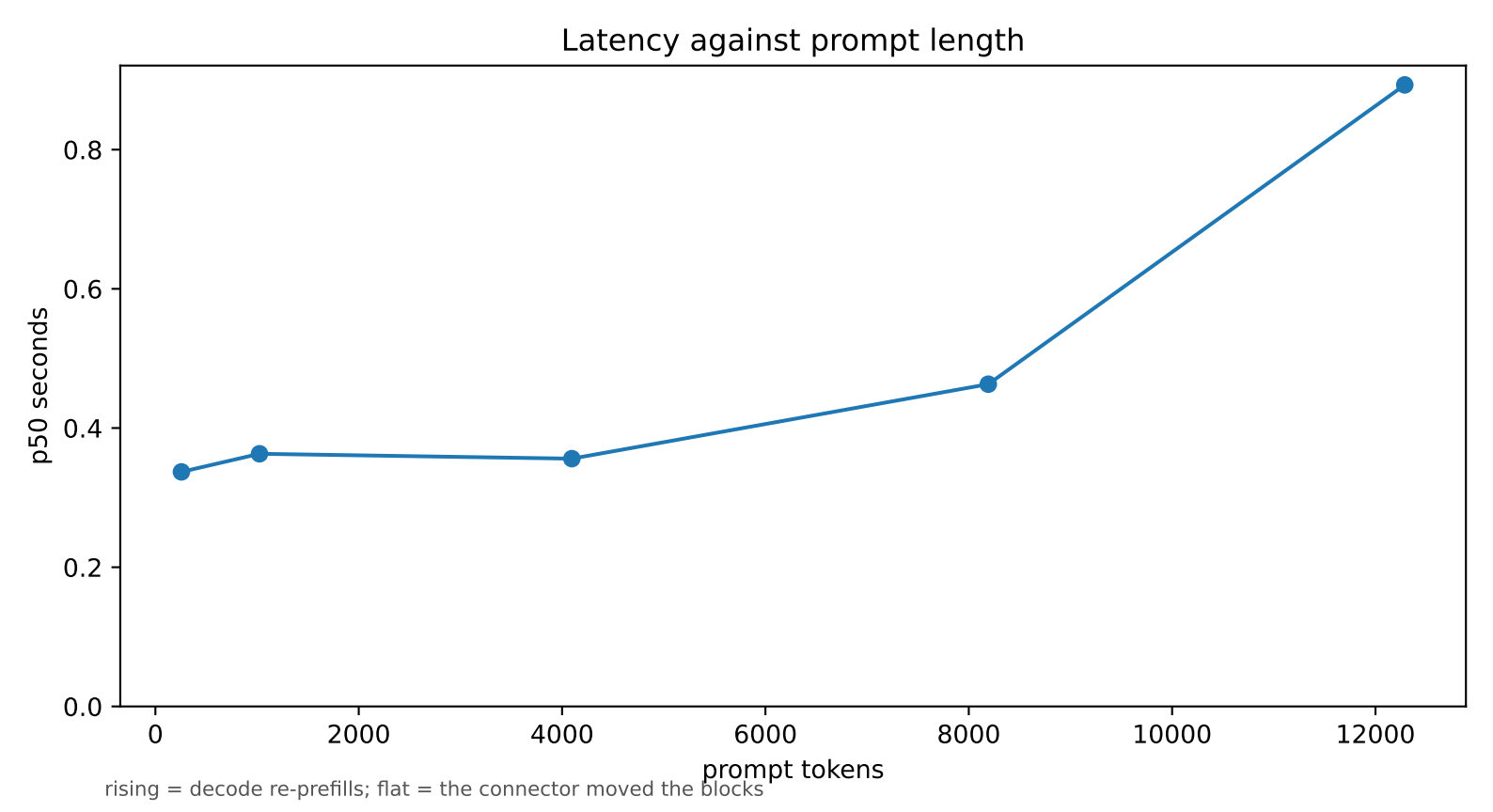

Where the 12 GiB slice goes
GiB
0
1
2
3
4
5
weights
activations + graphs
KV cache
only the third bar holds sequences


Where decode gets its prompt KV from
prompt tokens
0
100000
200000
300000
400000
500000
600000
computed from scratch
decode's own prefix cache
from the connector
only the third bar is a KV hop; the second is vLLM caching its own repeats


KV usage against the admission threshold
ratio
0.05
0.04
0.03
0.02
0.01
0.00
prefill
decode
dashed line is the kv_threshold trigger



flat is healthy; a rising line means something accumulates


Latency against prompt length
p50 seconds
0.0
0.2
0.4
0.6
0.8
0
2000
4000
6000
8000
10000
12000
prompt tokens
rising = decode re-prefills; flat = the connector moved the blocks